

```

{
    nthreads = omp_get_num_threads();
    printf("Number of threads = %d\r", nthreads);
}

/* All threads join master thread and disband */
}

```

With work-sharing constructs, OpenMP splits the work of parallel regions and distributes them among threads. OpenMP work-sharing constructs are:

- *loop* (for a loop in C/C++)
- *section*
- *single*

Parallelising loops are the most important constructs in OpenMP. In your code, you need to find the most time-consuming loops, and you can split the task inside it among the team of threads.

See the following snippet of code:

```

int a[5000], i;
#pragma omp parallel for
for(i=0; i<5000; i++)
{
    a[i] = i;
}

```

The above snippet demonstrates how to parallelise a simple loop using the loop construct. The loop iteration value (variable 'i') is private, by default, to each thread in the team, and each thread is assigned a fixed-size chunk. It is important to note that the loop in the work-sharing region must be iteration-independent, so that each thread can safely execute its chunk independently -- in any order without carrying any previous loop-dependency.

Second in work-sharing constructs, is *section*. With the *section* work-sharing directive, individual blocks of code are distributed among the threads in the team. Each thread can execute one or more sections and each section is executed exactly once. Some threads may even remain idle if the sections are too few. The following code snippet then gives an imprint of the section construct:

```

#pragma omp parallel sections
{
    #pragma omp section
    {
        x_cal();
    }
    #pragma omp section
    {
        y_cal();
    }
    #pragma omp section
    {

```

```

        z_cal();
    }
}

```

The third in work-sharing constructs is *single*. The *single* construct denotes a block of code that is executed by any one thread from the pool of threads, including the master thread. The other threads in the team, which do not execute the block, simply wait at the implicit barrier at the end of *single* construct, unless a *nowait* clause is specified. One possible use of this construct is to initialise shared variables.

```

#pragma omp single
{
    a=10; /* N-1 threads bypass this block */
}

```

How to do thread synchronisation

Well, the important thread synchronisation constructs are:

- **Master directive:** The block that is executed by only the master thread.
- **Critical directive:** The block that is executed by only a *single* thread when taking care of the race-condition.
- **Barrier directive:** An explicit barrier at a point at which the construct appears.
- **Taskwait directive:** Specifies a wait on the completion of the child task (generated at the beginning of the current task).
- **Atomic directive:** Ensures that a specific storage location is updated atomically.

OpenMP also provides a set of general-purpose, run-time library routines that can be used for synchronisation. These general-purpose routines operate on OpenMP lock variables. Some of these important routines are, *omp_init_lock()*, *omp_destroy_lock()*, *omp_set_lock()*, and *omp_unset_lock()*.

So, what's next?

We've discussed OpenMP here with the purpose of sharing its basic idea. You can further consult "OpenMP Application Program Interface", which you can download from the OpenMP official site (www.openmp.org). 

References

- OpenMP Application Program Interface Version 3: www.openmp.org/mp-documents/spec30.pdf
- OpenMP tutorial: computing.lnl.gov/tutorials/openMP
- Wikipedia on OpenMP: en.wikipedia.org/wiki/OpenMP

By: Vivek Kumar Prasad

The author is a system engineer at IBM India, and works for IBM-ISS (Internet Security Systems). He is based in Pune and you can reach him at mail_DOT_vivekkumar_AT_rediffmail_DOT_com.